0. **Code:** bhagam sai goutham.zip

1. final / final / Program.cs
2. final / insertion / Program.cs
3. final / largest frequency / Program.cs

1. **final / final / Program.cs**

```
namespace final
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int integer;
            int baseN;
            //Creating console prompts for integer and baseN
            Console.Write("Enter an integer: ");
            integer = int.Parse(Console.ReadLine());
            //baseN entered is positive re-prompt if negative or zero
            do {
                Console.Write("Enter the baseN :");
                baseN = int.Parse(Console.ReadLine());
            } while (baseN ≤ 0);
            int[] digits = BaseNdigits(integer, baseN);
            int sum = sumdigits(digits); //Calculate the sum of baseN digits
            Console.WriteLine($"sum of base digits={sum}");
            int maxdigit = findmaxdigit(digits); //find largest baseN digits
            Console.WriteLine($"{maxdigit} is the largest digit");

        }
        //break-down the integers and fetching remainders
        static int[] BaseNdigits(int integer, int baseN)
        {
            int digitcnt = 0;
            int temp = integer;
            while (temp > 0) //count the number of digits
```

A1

A2

A4

A5

if(integer <0) integer = -integer; A3

1. **final / final / Program.cs**

```
            {
                temp /= baseN;
                digitcnt++;
            }
            int[] digits = new int[digitcnt];
            temp = integer;
            for (int i = digitcnt - 1; i ≥ 0; i--) //extract digits from right to left
            {
                digits[i] = temp % baseN;
                temp /= baseN;
            }
            return digits;
        }
        //calculates the sum of all the integers
        static int sumdigits(int[] digits)
        {
            int sum = 0;
            int i = 0;
            while (i < digits.Length)
            {
                sum += digits[i];
                i++;
            }
            return sum;
        }
        //fetches the largest integer
        static int findmaxdigit(int[] digits)
        {
            int maxdigit = digits[0];
            int i = 1;
```

Could use new List<int>()

While loop.

A3

Separate

Not necessary thro separate problems.

A4

if(digits.Length == 0) Leave Another one can be empty. when digits = 0.

A5

1. **final / final / Program.cs**

```
            while (i < digits.Length)
            {
                if (digits[i] > maxdigit)
                {
                    maxdigit = digits[i];
                }
                i++;
            }
            return maxdigit;
        }


    }
}
```

A5

2. final / insertion / Program.cs

```
namespace insertion
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int[] arr = {
                23,
                1,
                10,
                5,
                2
            };
            int[] countarr = countinsertion(arr);

            Console.WriteLine("Insertion counts for each element:");
            for (int i = 0; i < countarr.Length; i++)
            {
                Console.WriteLine(countarr[i]);
            }
        }
        //creating a method
        static int[] countinsertion(int[] arr)
        {
            int[] countarr = new int[arr.Length];
            for (int i = 0; i < arr.Length; i++) // iterate the each
              element
            {
                int insertcount = 0;
                for (int j = 0; j < i; j++)
                {
```

B1

B1

B2

2. final / insertion / Program.cs

```
                if (arr[j] > arr[i])
                {
                    insertcount++;
                }
            }
            countarr[i] = insertcount;
        }
        return countarr; // returns the count array of insertions
    }
    /*
    static int[] countinsertions(int[] arr)
    {
        int[] countarr= new int[arr.Length];
        for(int i = 1; i < arr.Length; i++)
        {
            int key = arr[i];
            int j = i - 1;
            int count = 0;
            while(j≥0&& arr[j] > key)
            {
                count++;
                j--;

            }
            countarr[i] = count;
        }
        return arr;
    }
    static void printarray(int[] arr)
    {
        int i= 0;
```

B3

B4

2. final / insertion / Program.cs

```
            while(i<arr.Length)
            {
                Console.WriteLine(arr[i]+ " ");
                i++;
            }
            Console.WriteLine();
        }
        */
    }
}
```

3. final / largest frequency / Program.cs

```
namespace largest_frequency
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int[] arr = {
                1,
                2,
                3,
                2,
                7,
                3,
                4,
                4,
                5,
                5,
                4,
                5,
                1,
                6,
                1,
                1
            };
            int sum = largestnumberfrequency(arr);
            Console.WriteLine($"Returned sum: {sum}");

        }
        static int largestnumberfrequency(int[] arr)
        {
            if (arr == null || arr.Length == 0)
```

3. **final / largest frequency / Program.cs**

```
            {
                return 0;
            }
            Dictionary < int, int > frequencydict = new Dictionary < int, int > ();
            Dictionary < int, int > largestvaluedict = new Dictionary < int, int > ();
            int i;
            for (i = 0; i < arr.Length; i++)
            {
                int num = arr[i];
                if (frequencydict.ContainsKey(num))
                {
                    frequencydict[num]++;
                }
                else
                {
                    frequencydict[num] = 1;
                }

                int frequency = frequencydict[num];
                if (largestvaluedict.ContainsKey(num) || num > largestvaluedict[frequency])
                {
                    largestvaluedict[frequency] = num;
                }
                i++;
            }
            int sum = 0;
            i = 0;

```

C5

C2

C3

Separate loop — You can only get the largest frequency after you have gotten the count dict.

3. **final / largest frequency / Program.cs**

```
            while (i < largestvaluedict.Count)
            {
                int frequency = largestvaluedict[i];
                int value = largestvaluedict[i];
                sum += value * frequency;
                i++;
                //sum += largestvaluedict[i];
            }

            return sum;
        }
    }
}
```

C4

C5

okay except you shouldn't do it in parallel.